# Práctica de la Unidad II

# Estructuras de datos básicas

## C# / Windows Forms / .NET 10

Institución: Universidad Tecnológica Metropolitana

Asignatura: Estructura de Datos

Docente: Ing. Mirian Magaly Canché Caamal

Integrantes: Pendiente de completar por el equipo

Grupo: Pendiente de completar

Entrega: 23 de octubre de 2026

Versión C#: 23 de octubre de 2026

Doce ejercicios, tres proyectos independientes y un menú general. Las capturas de este informe provienen de los controles reales de la versión C#.

### Uso y arquitectura

Abrir Practica_U2_ED032026.sln o ejecutar INICIAR.cmd. COMPILAR_Y_PROBAR.cmd compila y verifica los cuatro proyectos. Las clases de Modelos.cs resuelven la lógica; Program.cs contiene la interfaz; Pruebas.cs valida modelos y botones.

Convención de nombres: el formulario es frmPractica y cada control lleva prefijo por tipo y nombre descriptivo: btnRetirar, txtClaveRetiro, lblEstadoL1, dgvTablaC1, cboMovimiento, pnlCoches, tabC2, mnuSalir. Las pruebas verifican que todos los nombres tengan prefijo y sean únicos. Cada pestaña muestra un ícono de su estructura (lista, pila o cola).

Requisitos: Windows y SDK .NET 10 para compilar. Los ejecutables publicados requieren .NET Desktop Runtime 10. Los datos se mantienen durante la sesión; el récord del juego se conserva en LocalAppData.

# L1. Productos disponibles y retirados

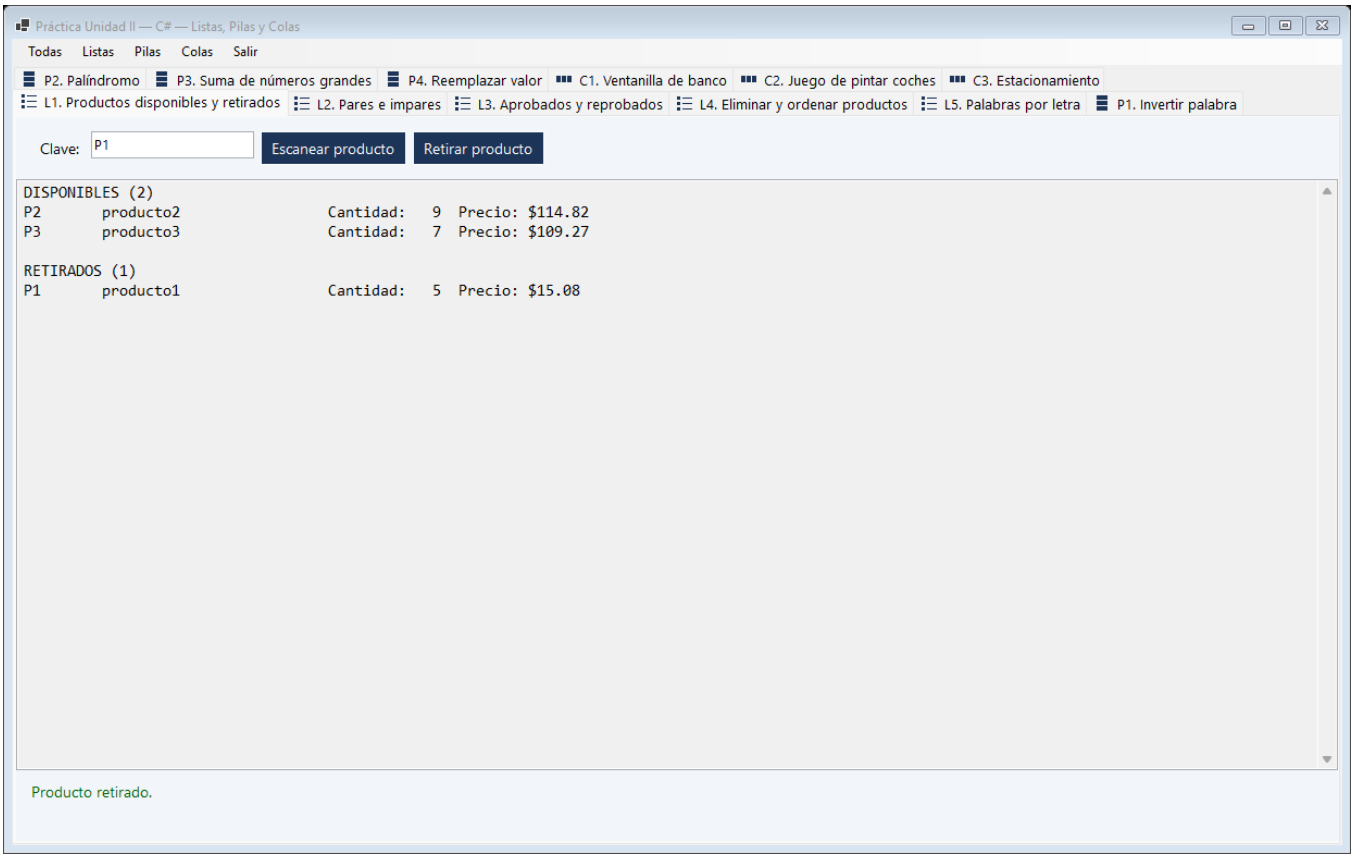
Problema: Administrar el almacén de productos escaneados y conservar los retirados.

Entradas: Clave del producto a retirar; la cantidad y el precio se generan automáticamente.

Proceso: Escanear crea un objeto Producto y lo agrega a List<Producto>. Retirar busca por clave y mueve el objeto a otra lista.

Salidas: Productos disponibles y retirados, con cantidades, precios y número de elementos.

Funcionamiento y evidencia: Pulsa Escanear producto dos veces; escribe P1 y pulsa Retirar producto. La imagen muestra el producto1 retirado y el producto2 disponible. Una clave inexistente produce un mensaje sin perder datos.



# L2. Pares e impares

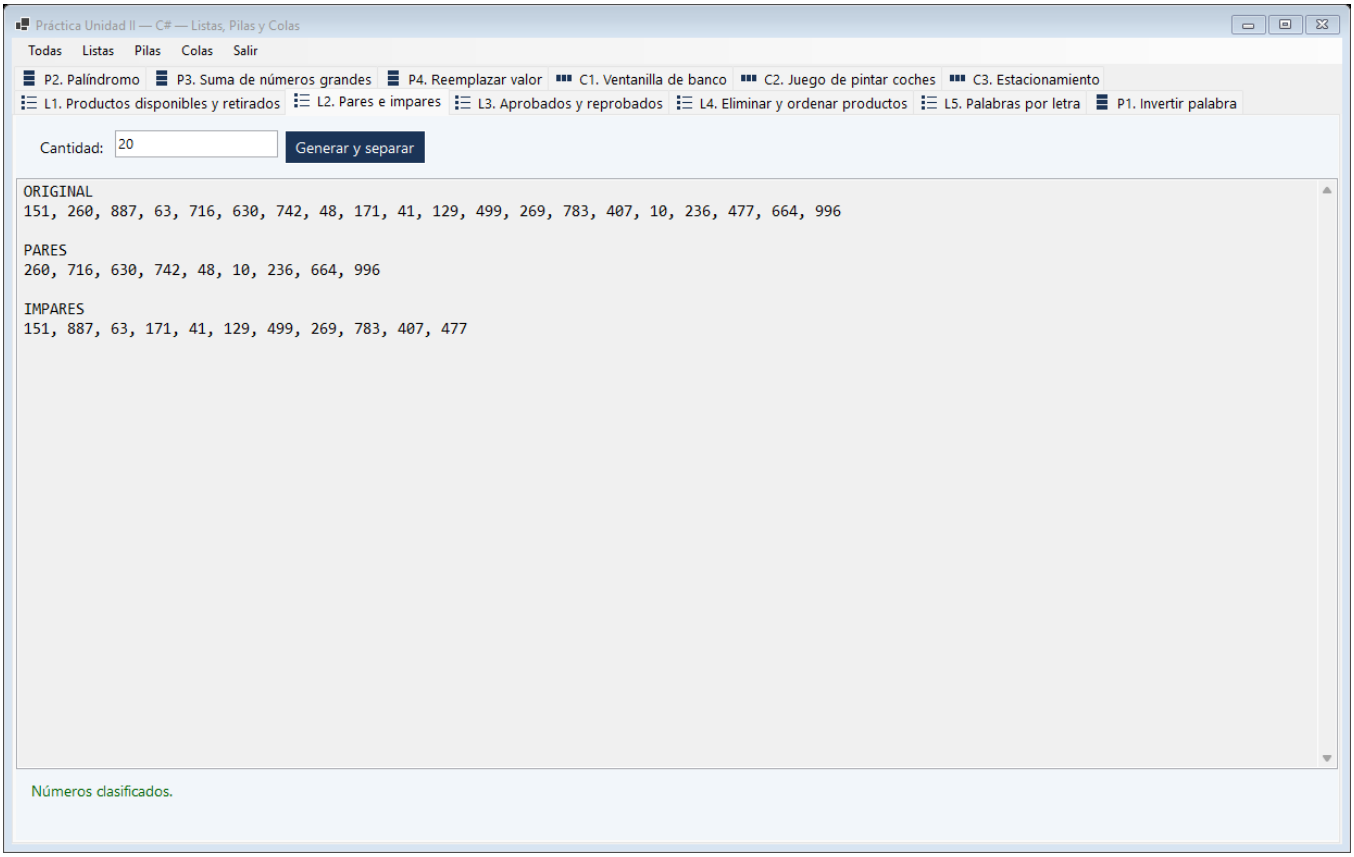
Problema: Separar una lista de números aleatorios en listas por paridad.

Entradas: Cantidad entera entre 1 y 500.

Proceso: Genera números de 0 a 999 y usa el residuo entre dos para agregarlos a las listas de pares e impares.

Salidas: Lista original y las dos listas clasificadas.

Funcionamiento y evidencia: Introduce 20 y pulsa Generar y separar. La captura muestra las tres listas; ningún número se pierde. Cero, decimales y letras se rechazan como cantidad.



# L3. Aprobados y reprobados

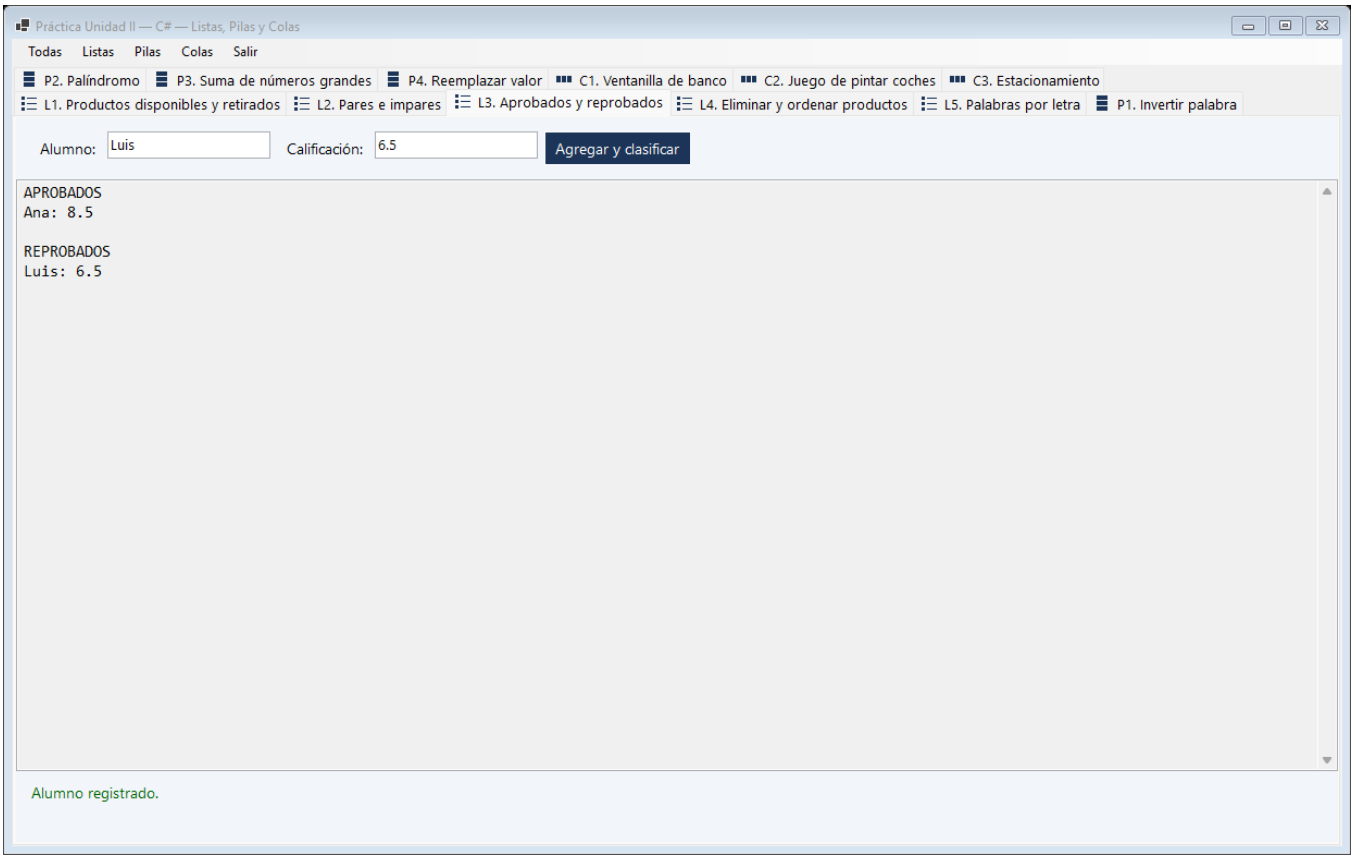
Problema: Clasificar alumnos según su calificación.

Entradas: Nombre y calificación decimal de 0 a 10.

Proceso: Agrega al alumno y extrae de la lista inicial los que tengan calificación menor que siete, trasladándolos a reprobados.

Salidas: Listas de aprobados y reprobados con sus notas.

Funcionamiento y evidencia: Registra Ana con 8.5 y Luis con 6.5. La captura muestra a cada alumno en su lista. Una nota igual a 7 aprueba; un nombre vacío o nota fuera de rango genera error.



# L4. Eliminar y ordenar productos

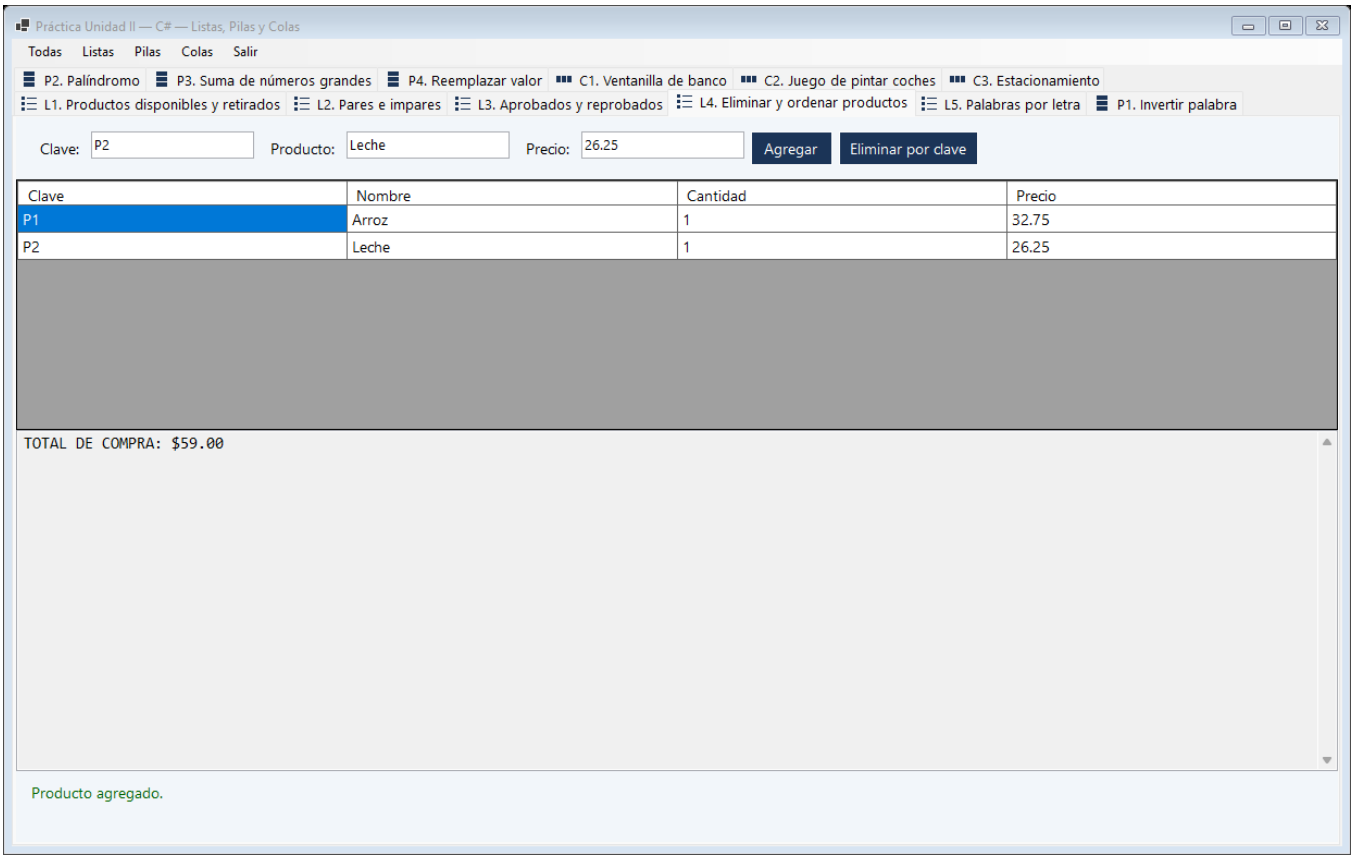
Problema: Mantener una compra con eliminación por clave y total.

Entradas: Clave única, nombre y precio decimal no negativo.

Proceso: Agrega a una lista, elimina por clave, ordena por nombre con comparación en español y suma precios usando decimal.

Salidas: Tabla ordenada y costo total.

Funcionamiento y evidencia: Agrega Arroz de 32.75 y Leche de 26.25. La imagen muestra total 59.00. Para eliminar, escribe la clave y pulsa Eliminar por clave. Se rechazan claves duplicadas y precios negativos.



# L5. Palabras por letra

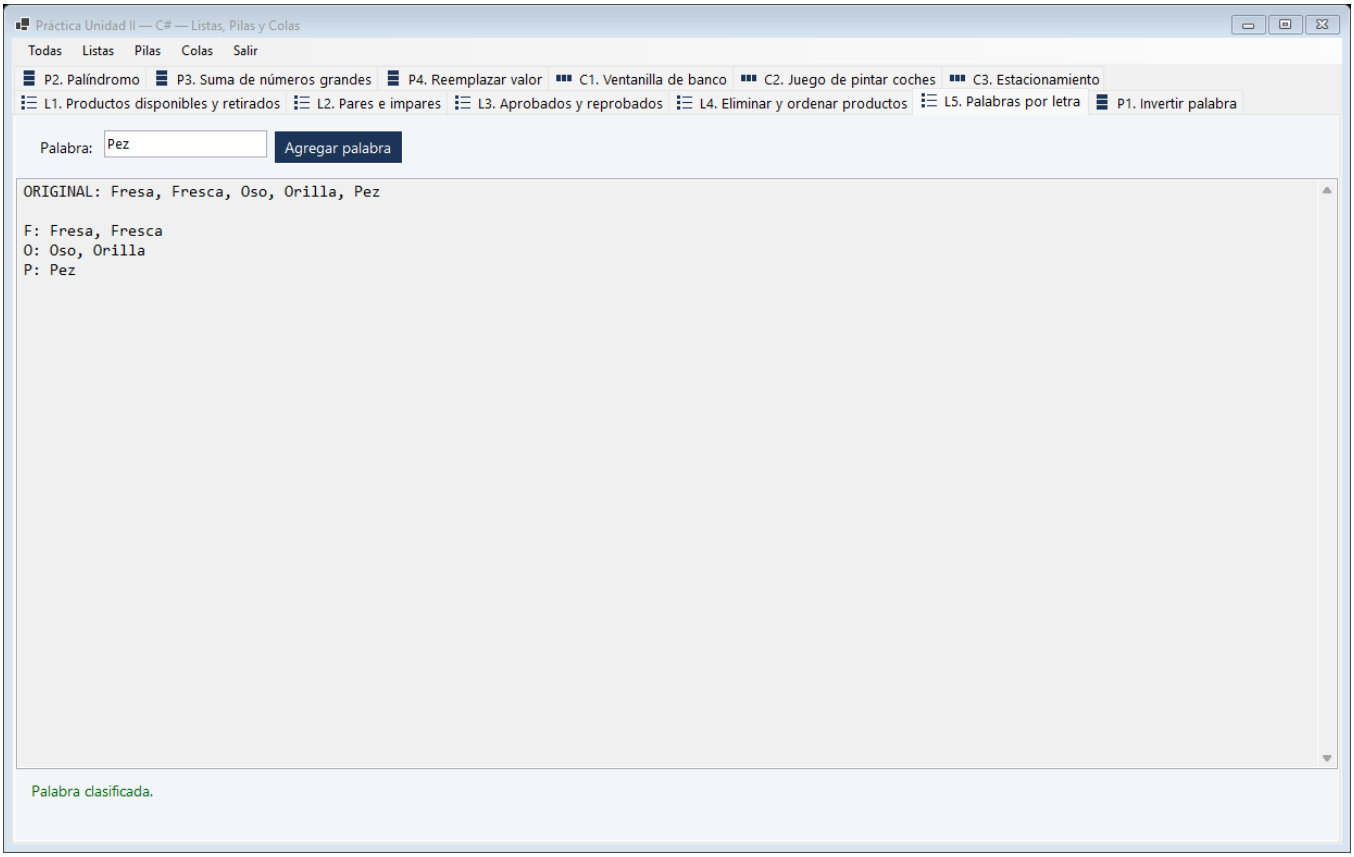
Problema: Crear listas dinámicas según la primera letra de cada palabra.

Entradas: Una palabra formada por letras en cada captura.

Proceso: Conserva la lista original y agrupa por inicial en un diccionario ordenado cuyos valores son List<string>.

Salidas: Lista original y una lista por inicial.

Funcionamiento y evidencia: Agrega Fresa, Fresca, Oso, Orilla y Pez. La captura muestra grupos F, O y P. Nuevas iniciales crean listas automáticamente; se respetan letras acentuadas.



# P1. Invertir palabra

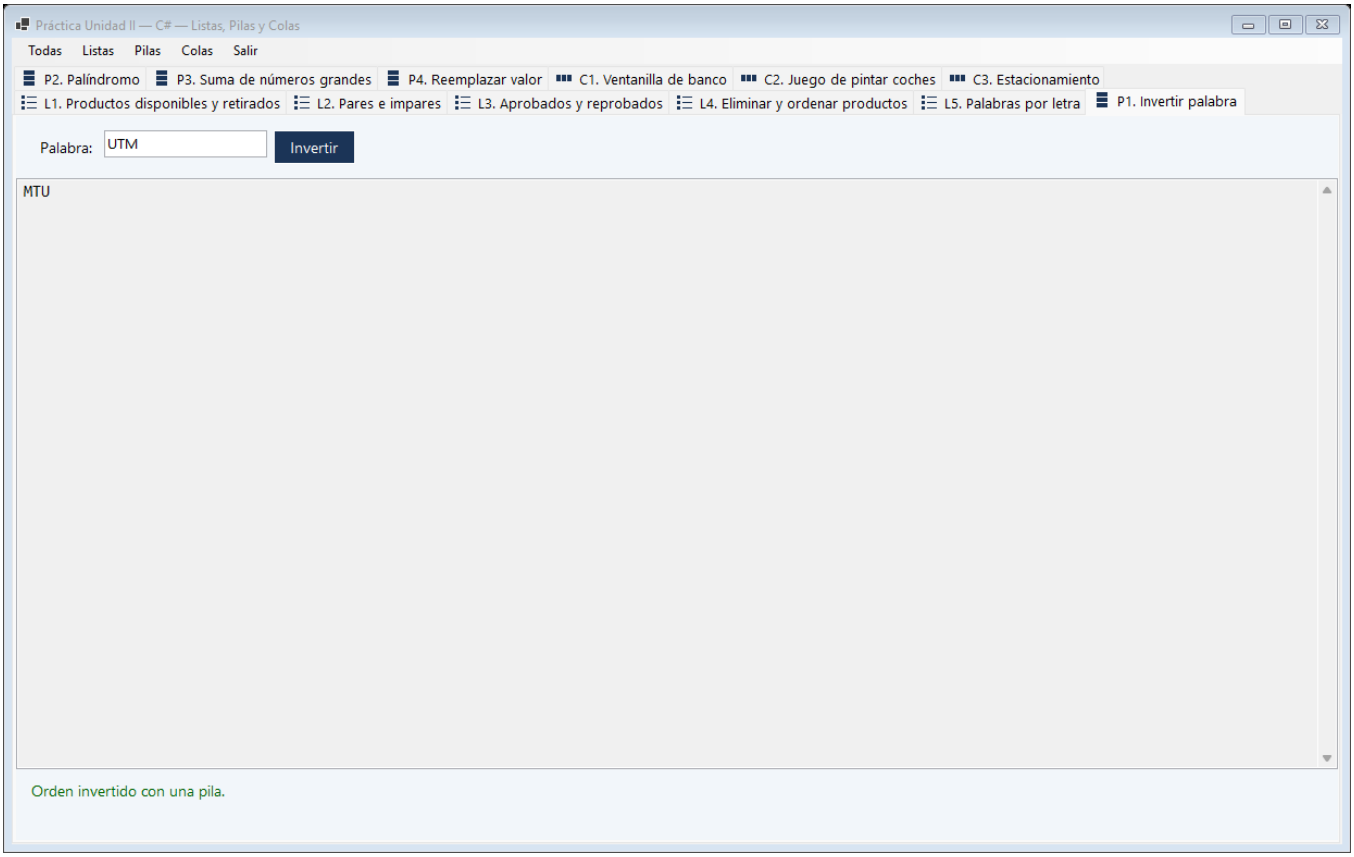
Problema: Mostrar una palabra en orden inverso mediante una pila.

Entradas: Palabra no vacía.

Proceso: Apila cada Rune del texto y desapila hasta vaciar la estructura.

Salidas: Texto invertido, preservando los caracteres Unicode completos.

Funcionamiento y evidencia: Escribe UTM y pulsa Invertir. La captura muestra MTU. El texto vacío genera un mensaje de datos faltantes.



# P2. Palíndromo

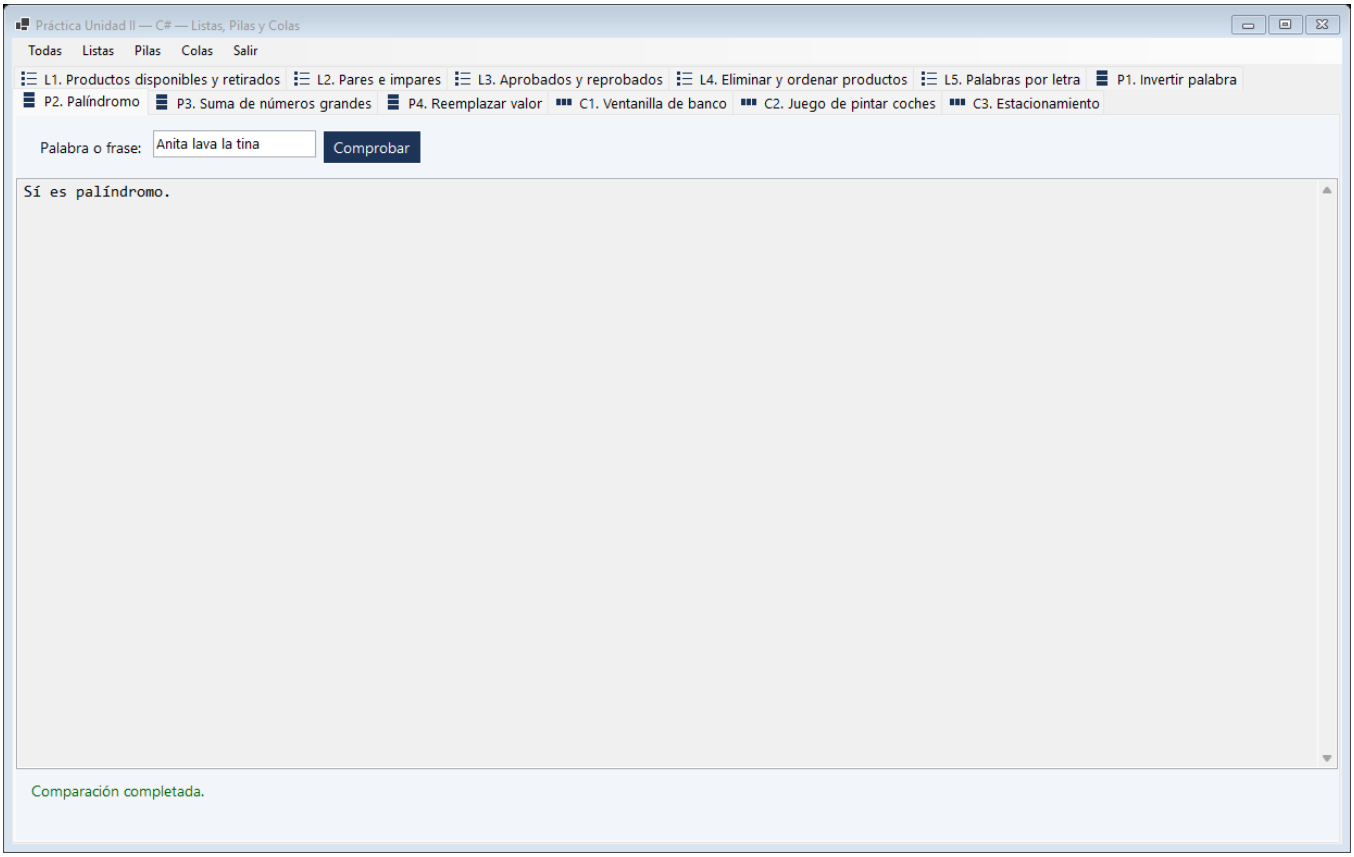
Problema: Determinar si un texto se lee igual en ambos sentidos.

Entradas: Palabra, frase o número.

Proceso: Normaliza mayúsculas y elimina espacios y signos; compara con la inversión realizada mediante pila.

Salidas: Mensaje afirmativo o negativo.

Funcionamiento y evidencia: Introduce Anita lava la tina y pulsa Comprobar. La captura muestra que es palíndromo. Se conservan acentos y se rechazan entradas compuestas únicamente por signos.



# P3. Suma de números grandes

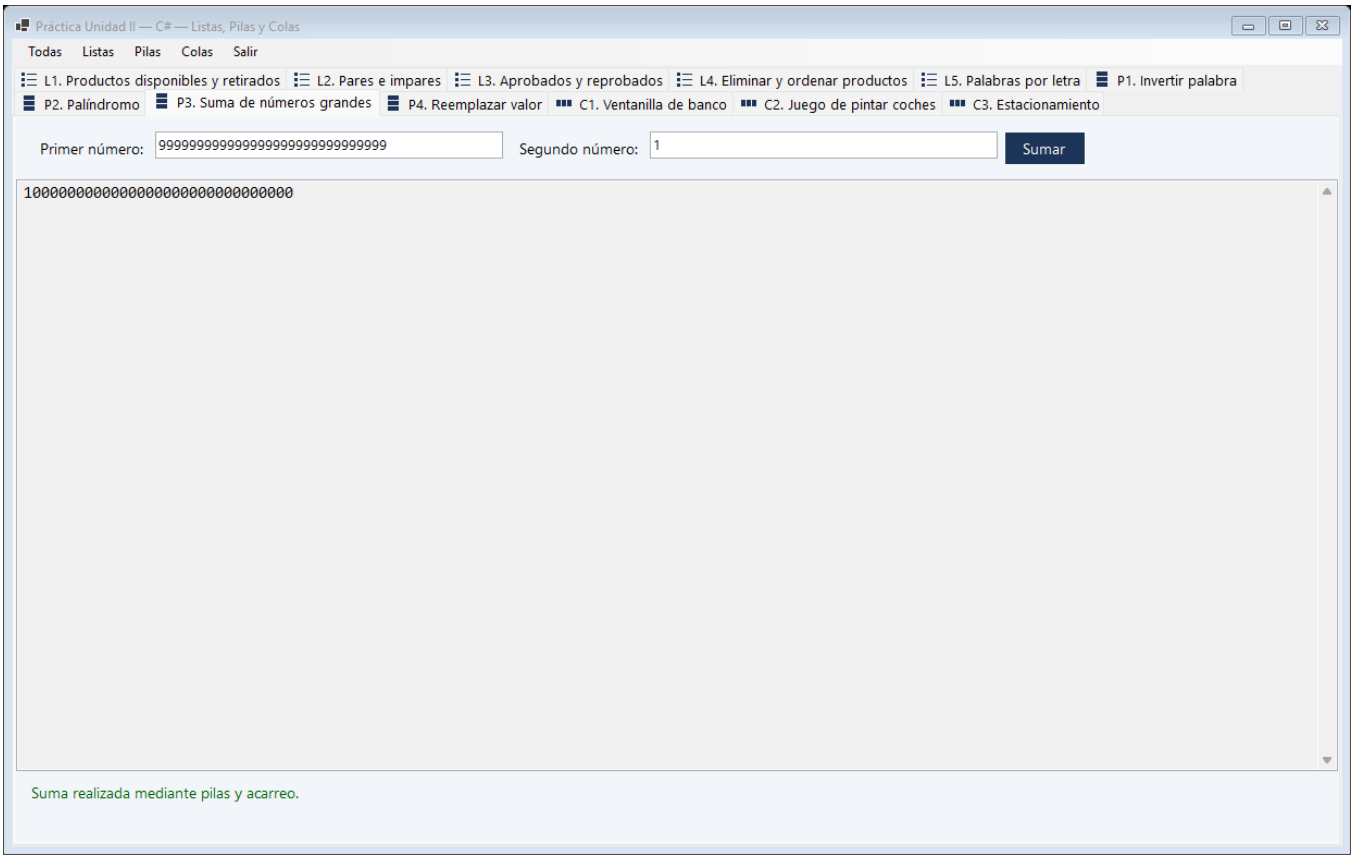
Problema: Sumar enteros que exceden los tipos numéricos ordinarios.

Entradas: Dos cadenas de dígitos, no negativas y sin separadores.

Proceso: Apila dígitos de ambos operandos, desapila de derecha a izquierda, suma con acarreo y construye el resultado con una tercera pila.

Salidas: Suma completa sin límite fijo de dígitos.

Funcionamiento y evidencia: Introduce treinta nueves y 1, y pulsa Sumar. La imagen muestra un uno seguido de treinta ceros. Letras y signos se rechazan. BigInteger sólo se usa para comprobar las pruebas.



# P4. Reemplazar valor

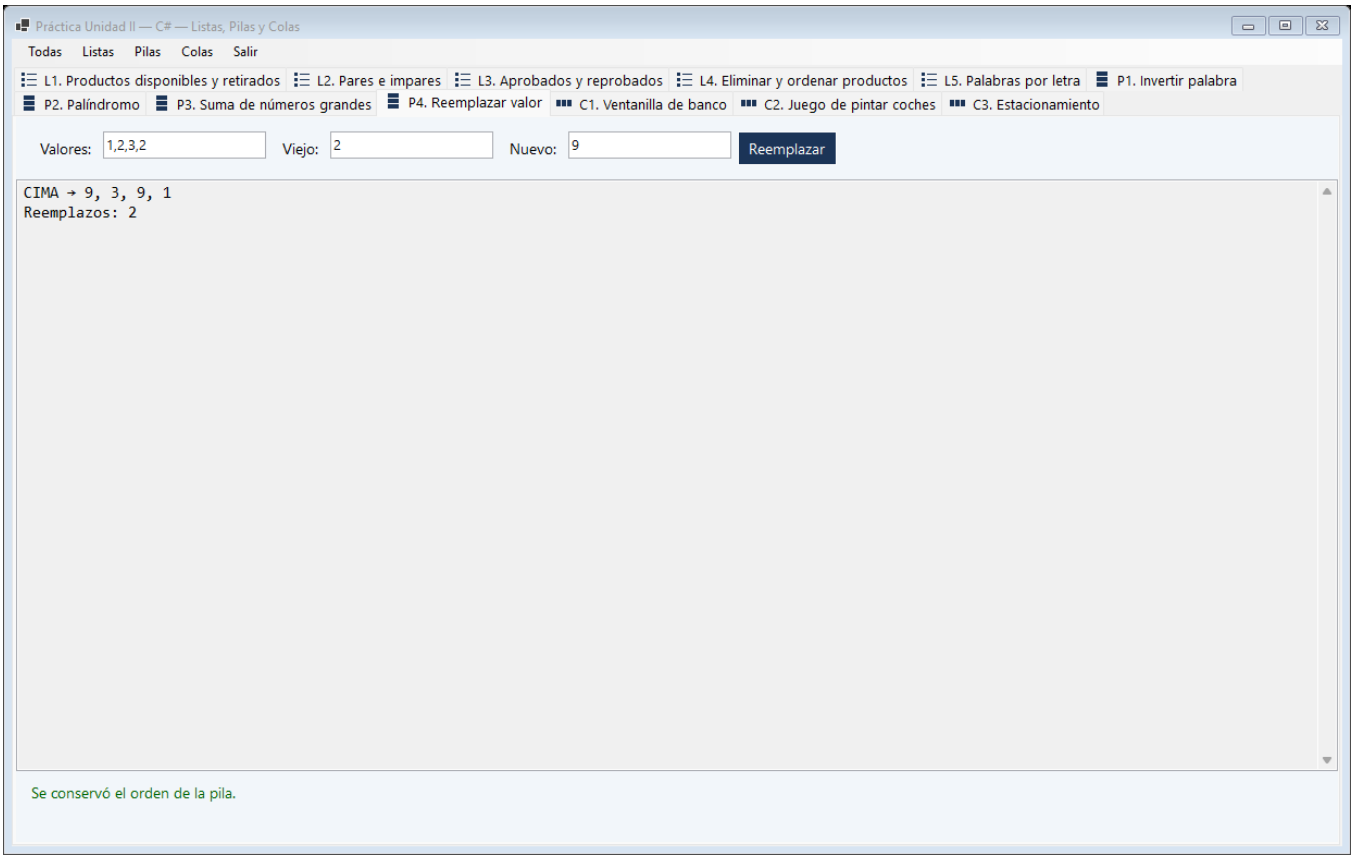
Problema: Sustituir un entero por otro en una pila sin alterar su orden.

Entradas: Enteros separados por comas, valor viejo y valor nuevo.

Proceso: Desapila hacia una pila auxiliar sustituyendo coincidencias; después restaura la pila original.

Salidas: Pila desde la cima y cantidad de reemplazos.

Funcionamiento y evidencia: Con valores 1,2,3,2, viejo 2 y nuevo 9, pulsa Reemplazar. La captura muestra 9,3,9,1 desde la cima y dos reemplazos. Se interpreta el error de redacción del enunciado como reemplazar viejo por nuevo.



# C1. Ventanilla de banco

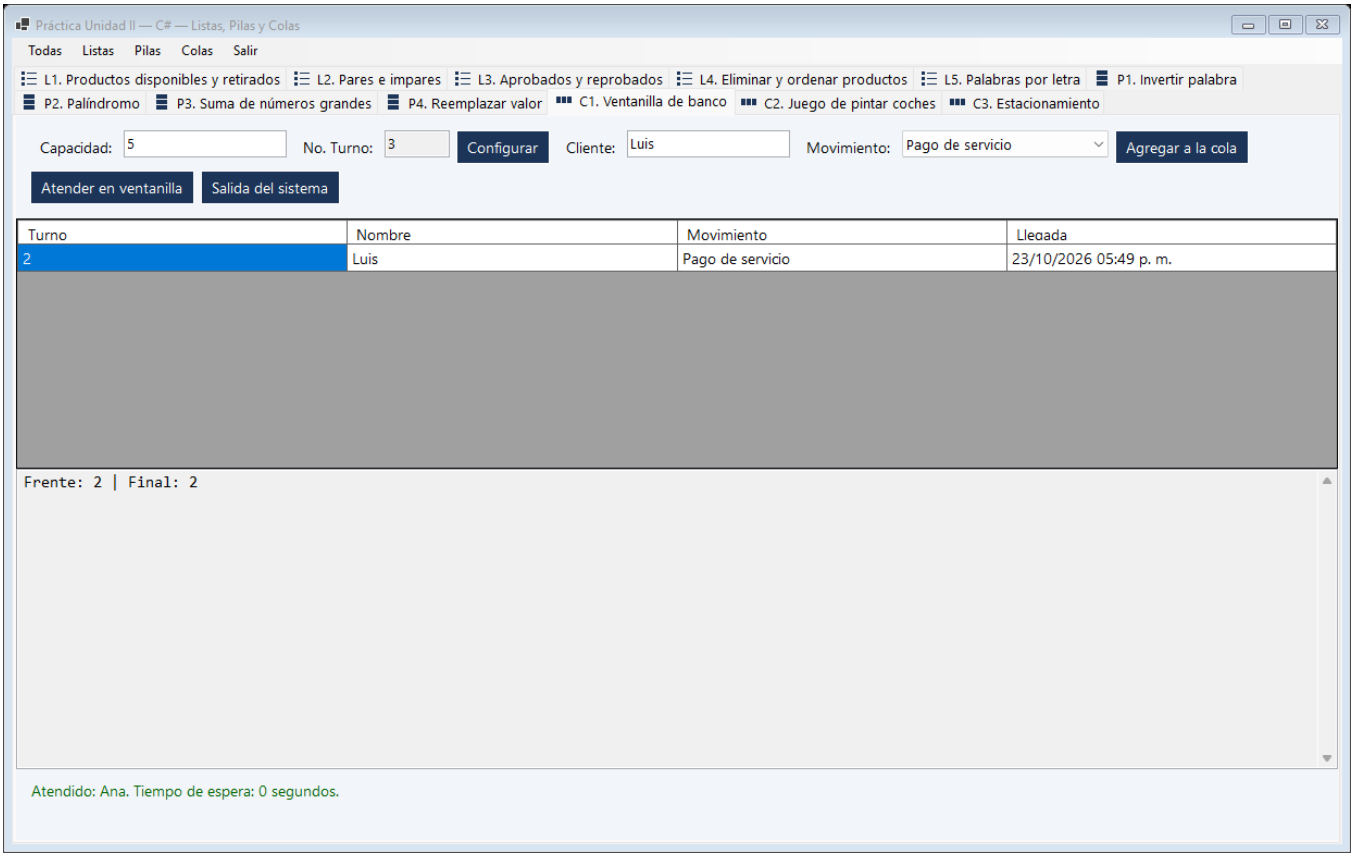
Problema: Administrar clientes mediante cola FIFO con capacidad limitada.

Entradas: Nombre, movimiento del ComboBox y capacidad configurable con cola vacía.

Proceso: Asigna turno y hora DateTime.Now; InsertaCola valida ColaLlena. Atender usa EliminaCola tras validar ColaVacia y calcula la espera.

Salidas: DataGridView actualizado, No. Turno siguiente, frente y final; ventana "SE AGREGA UN CLIENTE AL FINAL DE LA COLA" con turno, nombre, movimiento y hora de llegada; ventana "TIEMPO DE ESPERA EN LA COLA" al atender.

Funcionamiento y evidencia: Registra Ana y Luis y atiende a Ana. Cada alta abre la ventana de confirmación y la atención abre la ventana con el tiempo de espera. La captura deja a Luis en la cola. La capacidad inicial es cinco; se validan sobreflujo y atención con cola vacía. El botón Salida del sistema cierra la aplicación.



# C2. Juego de pintar coches

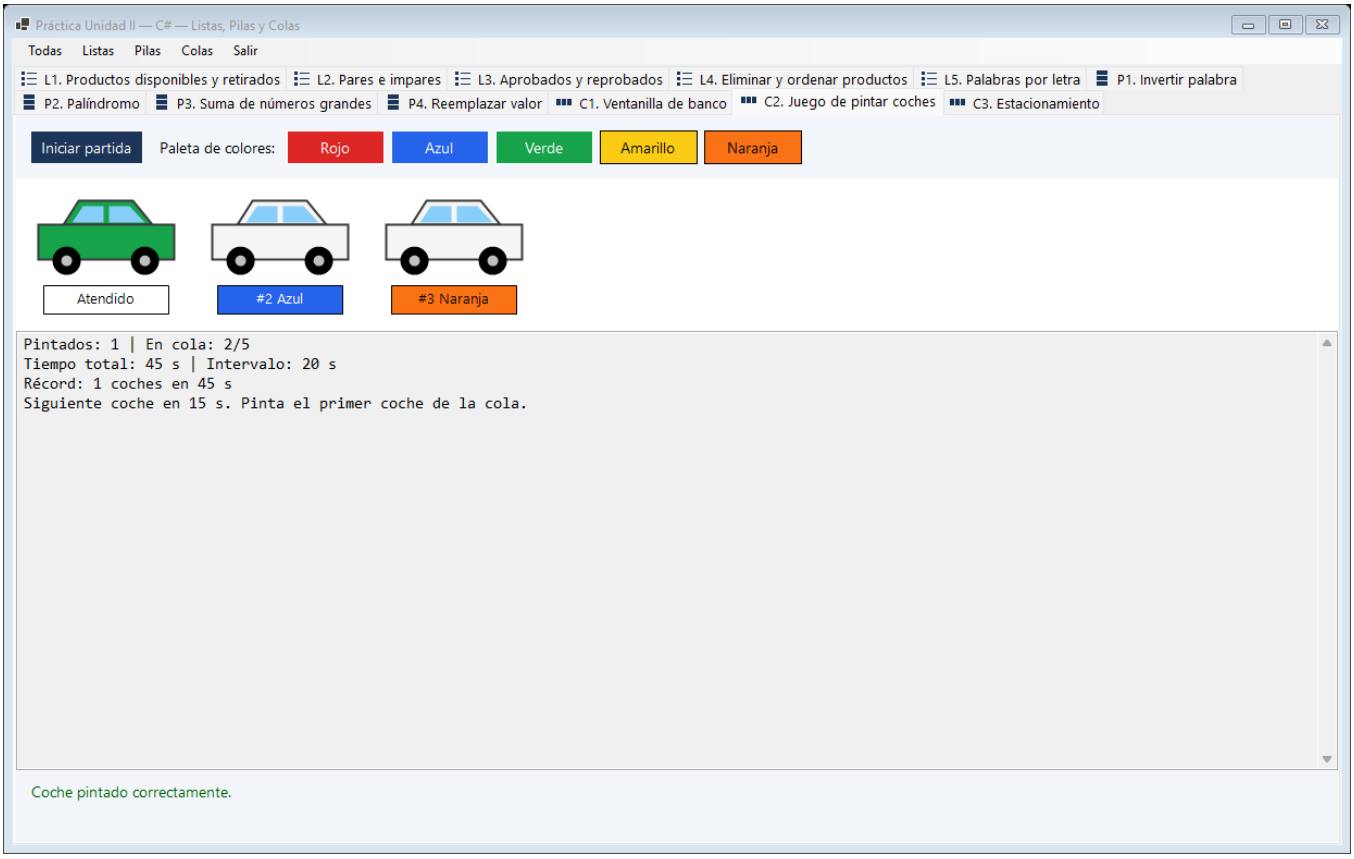
Problema: Atender coches en orden y pintarlos del color solicitado antes de que se acumulen cinco.

Entradas: Botón Iniciar partida y selección de color en la paleta de botones coloreados.

Proceso: Un Timer actualiza el modelo con un reloj monótono. El primer coche aparece inmediatamente; los siguientes cada 20 segundos. Cada tres aciertos reduce dos segundos el intervalo, hasta un mínimo de dos.

Salidas: Coches en cola dibujados sin pintar con una etiqueta del color solicitado; el último coche atendido aparece pintado con su color. Puntuación, tiempo, cuenta regresiva del siguiente coche y récord.

Funcionamiento y evidencia: Pulsa Iniciar partida y, en la paleta, el color de la etiqueta del primer coche. La captura muestra 45 segundos de partida: el coche atendido ya pintado y dos coches esperando con su etiqueta de color. Un color incorrecto no elimina el coche. Al llegar a cinco finaliza y se guarda el récord en LocalAppData.



# C3. Estacionamiento de autos

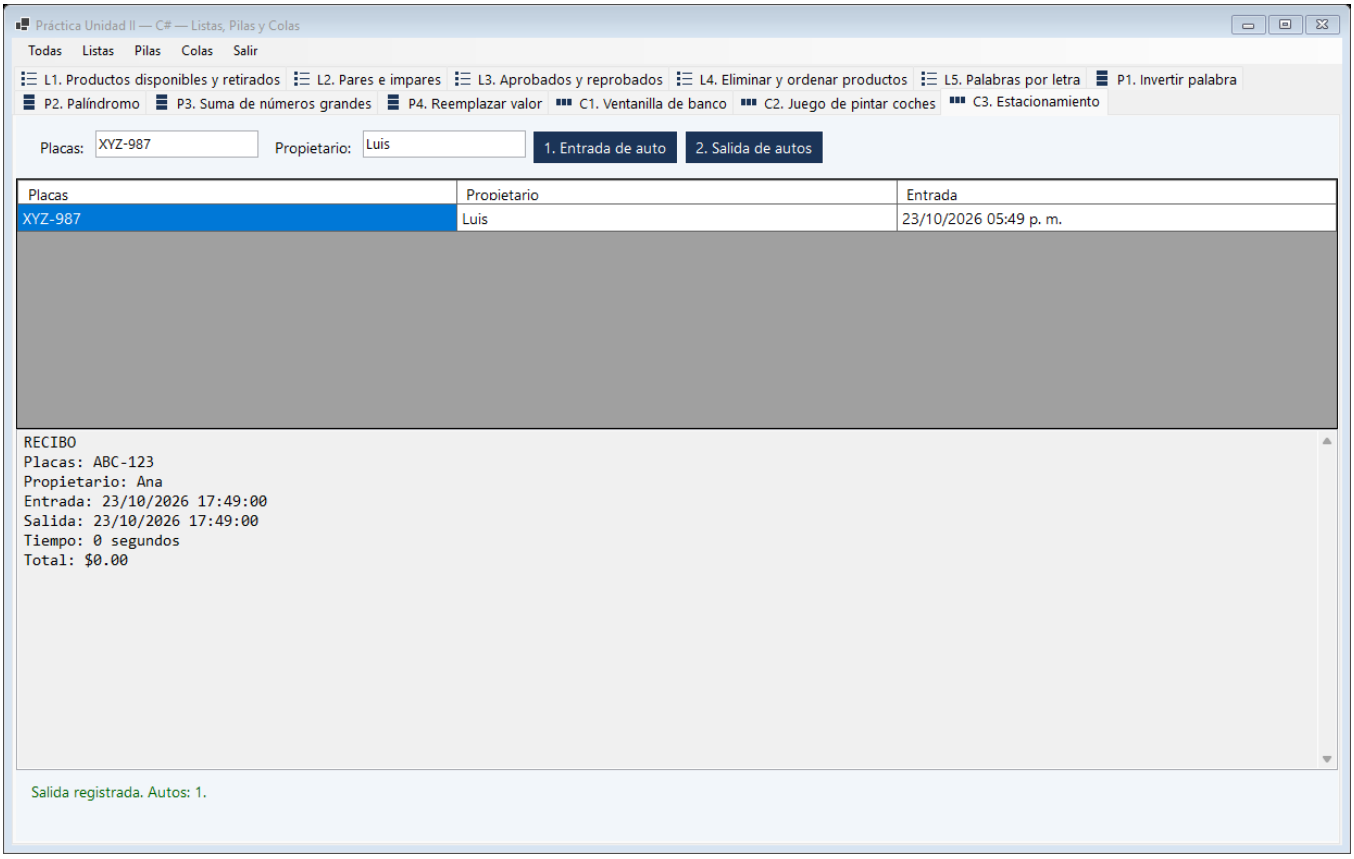
Problema: Controlar autos en un estacionamiento sin cupo fijo y cobrar dos pesos por segundo completo.

Entradas: Placas y propietario; hora de entrada y salida tomadas del sistema.

Proceso: Inserta cada auto como nodo de una cola circular doblemente ligada. Salir retira el frente, repara enlaces y calcula segundos transcurridos por 2.00.

Salidas: Ventana "ENTRADA DE AUTO" que confirma cada ingreso; tabla de autos y ventana "SALIDA DE AUTO" con placas, propietario, horas, tiempo y costo.

Funcionamiento y evidencia: Registra ABC-123 y XYZ-987 con la opción 1. Entrada de auto; cada ingreso se confirma con un mensaje. Pulsa 2. Salida de autos: la captura muestra el recibo del primero y el segundo todavía en cola. Se rechazan placas repetidas y salida con estacionamiento vacío.



# Validación y pruebas

La suite verifica rangos, decimales, listas, orden de las pilas, colas vacías y llenas, tiempos controlados, récord y enlaces circulares. Compara 200 sumas de números grandes con BigInteger y recorre mil nodos de estacionamiento. También ejecuta los flujos de los doce ejercicios mediante sus botones antes de capturar evidencias.

OK | 1. L1: productos aleatorios y rangos

OK | 2. L1: retiro e historial

OK | 3. L1: retiro duplicado

OK | 4. L2: clasificación conserva todos los números

OK | 5. L2: cantidad inválida

OK | 6. L3: límite de aprobación en siete

OK | 7. L3: calificación inválida

OK | 8. L4: orden y total decimal

OK | 9. L4: eliminación por clave

OK | 10. L4: clave duplicada

OK | 11. L5: listas dinámicas y acentos

OK | 12. L5: valida palabra

OK | 13. P1: inversión y Unicode

OK | 14. P2: frases, números y negativo

OK | 15. P2: signos sin letras

OK | 16. P3: acarreo de mil dígitos y cero

OK | 17. P3: 200 sumas contra BigInteger usado sólo en pruebas

OK | 18. P3: rechaza negativos

OK | 19. P4: reemplaza y conserva orden

OK | 20. P4: pila vacía

OK | 21. C1: cola vacía

OK | 22. C1: sobreflujo

OK | 23. C1: FIFO, frente, final y espera

OK | 24. C2: no llega antes de veinte segundos

OK | 25. C2: llegada exacta a veinte segundos

OK | 26. C2: color incorrecto

OK | 27. C2: coche atendido queda pintado con su color

OK | 28. C2: aceleración y récord

OK | 29. C2: fin con cinco coches y reloj detenido

OK | 30. C3: estacionamiento vacío

OK | 31. C3: placas únicas

OK | 32. C3: FIFO, cobro y enlaces

OK | 33. C3: mil nodos, circularidad y eliminación del último

OK | 34. Validación: coma decimal

OK | 35. Validación: entero

OK | 36. Validación: faltantes

OK | 37. Interfaz: doce ejercicios

OK | 38. Interfaz: nombres con prefijo y únicos

OK | 39. Interfaz L1: botón escanear

OK | 40. Interfaz: flujo completo L1. Productos disponibles y retirados

OK | 41. Interfaz: flujo completo L2. Pares e impares

OK | 42. Interfaz: flujo completo L3. Aprobados y reprobados

OK | 43. Interfaz: flujo completo L4. Eliminar y ordenar productos

OK | 44. Interfaz: flujo completo L5. Palabras por letra

OK | 45. Interfaz: flujo completo P1. Invertir palabra

OK | 46. Interfaz: flujo completo P2. Palíndromo

OK | 47. Interfaz: flujo completo P3. Suma de números grandes

OK | 48. Interfaz: flujo completo P4. Reemplazar valor

OK | 49. Interfaz: flujo completo C1. Ventanilla de banco

OK | 50. Interfaz: flujo completo C2. Juego de pintar coches

OK | 51. Interfaz: flujo completo C3. Estacionamiento

OK | 52. Interfaz C1: ventana al formar cliente

OK | 53. Interfaz C1: ventana con tiempo de espera

OK | 54. Interfaz C3: confirmación de entrada y recibo de salida

TOTAL: 54 verificaciones correctas. Sin fallos.